be able to use new language features even if browsers don't yet support them, the new features typically have a way to be transformed into older syntax. (This is what the Babel tool does—it converts newer JS features into older syntax for older browsers.)

esbuild claims to support "All modern JavaScript syntax," which is a little more vague than I like from my language tools, but in practice seems to mean that all new JavaScript language features that have reached stage 4 are supported. What that means is that you can freely use those features in your code and esbuild will be able to convert them to the browser-side version that you choose.

The version of code that esbuild emits to the browser is called the *target version*. By default, esbuild targets `esnext`, meaning it does not convert newer language features to support older browsers. You can change that setting by changing the `target` in the CLI command or in the configuration file (more on that in a moment). You can specify a specific ECMAScript version year as a target, as in `es2020`. The full set of rules as for what language features esbuild converts under what targets is part of the esbuild docs.[56] There are some other minor quirks to esbuild JavaScript support also described on that page.

esbuild converts `.ts` and `.tsx` files using TypeScript (and again, `.cts` and `.mts` files.) As mentioned in Chapter 6, *TypeScript*, the TypeScript compilation just removes the type annotations as though they were comments, and treats the file as though it was JavaScript. (There are a couple of TypeScript features, like enums, that are converted to JavaScript).

The esbuild TypeScript conversion works one file at a time, meaning TypeScript features that depend on cooperation between files that are not explicitly imported will not be compiled properly. In practice, this means that you need to be more explicit about importing types from another file.

Again, there are a couple of minor quirks to the esbuild functionality that you can read about in the esbuild docs.[57]

Those are the biggest and quirkiest cases. However, there are some other default